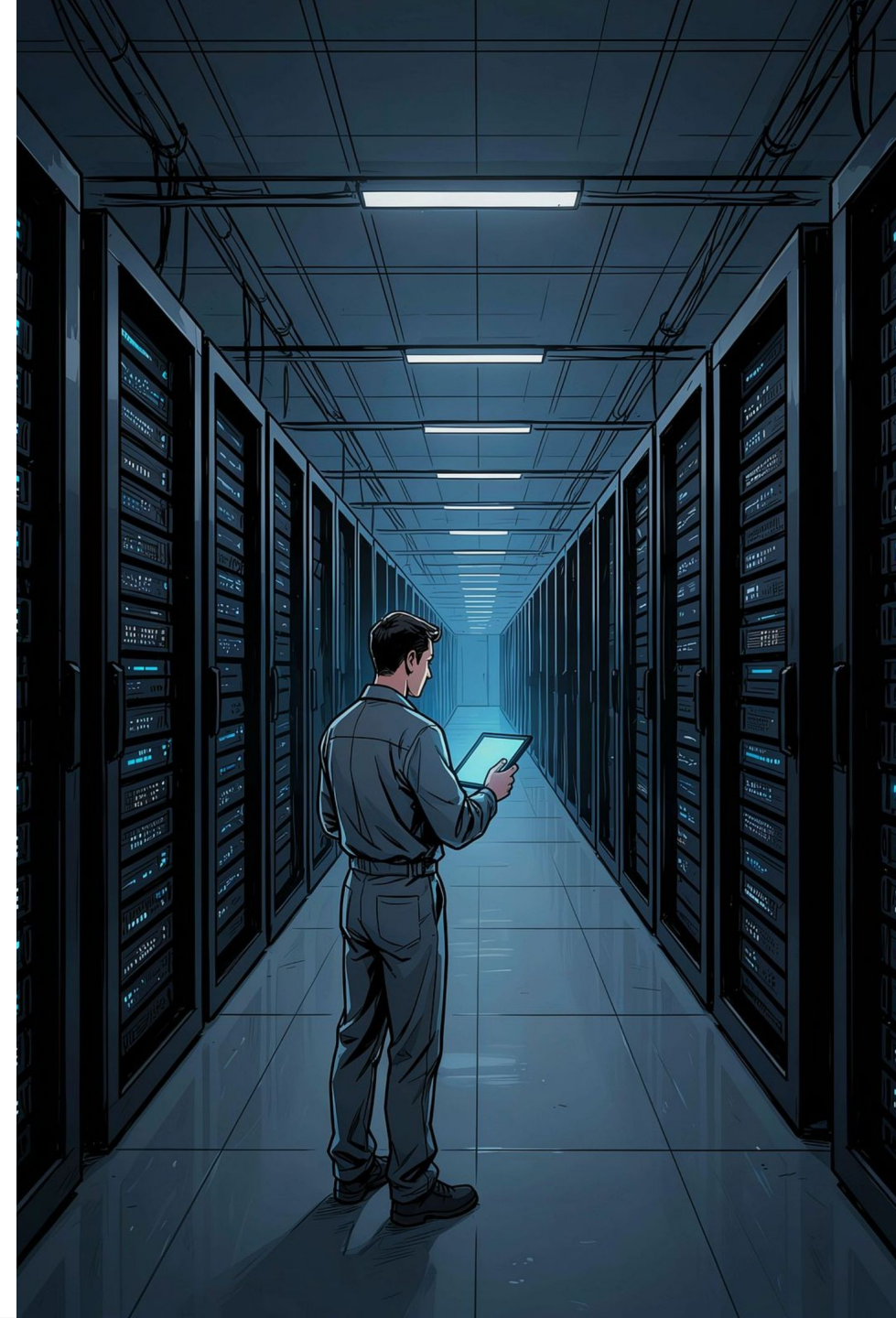


Capstone Project - Enterprise CI/CD Pipeline

Production Pipeline using **Jenkins** • **GitHub** • **Python Flask** • **Redis** •
Docker • **Docker Compose** • **Amazon ECR** • **SonarQube** • **Trivy**

DEVOPS CAPSTONE

ENTERPRISE GRADE



Project Overview

ABC Finance wants to modernize application deployment by automating the complete CI/CD lifecycle. The goal: build an automated, secure, and repeatable pipeline.

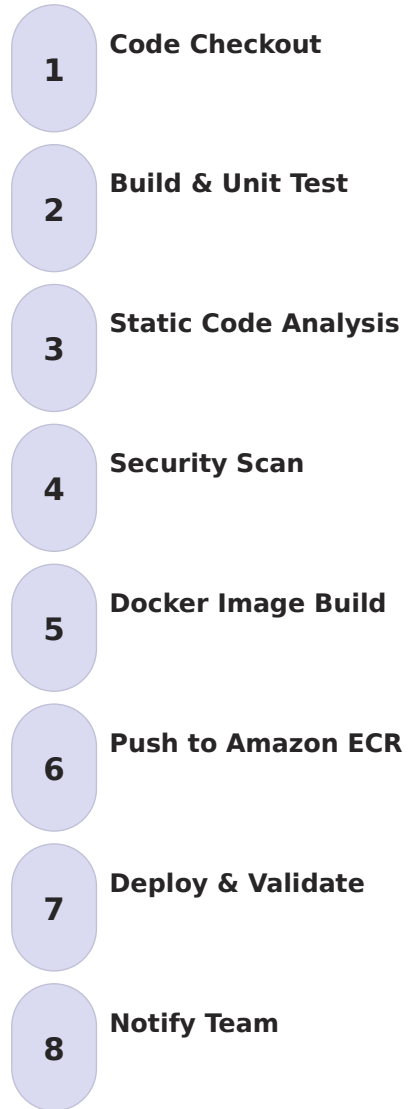
Current Problems

- **Manual deployments**
Error-prone, slow release cycles
- **No security scanning**
Vulnerabilities reach production
- **No code quality validation**
Technical debt accumulates unchecked
- **No centralized image registry**
Inconsistent container management



Business Requirements

Developers simply **push code to GitHub** — everything else happens automatically.



Solution Architecture

End-to-end enterprise pipeline — from developer commit to live containers serving users.



Every stage is automated and gated — no manual intervention required between commit and deployment.

Application Architecture

The application is a **Python Flask** web app backed by **Redis**, orchestrated via Docker Compose on a shared internal network.



Python Flask

Lightweight web framework serving the application API and UI layer.



Redis

In-memory data store for fast caching and session management.



Docker Network

Isolated bridge network enabling secure container-to-container communication.



Persistent Volume

Mounted volumes ensure Redis data survives container restarts.



Environment Variables

Secrets and config injected at runtime — never hardcoded in source.

Project Repository Structure

A clean, well-organized repo is the foundation of any enterprise pipeline. Every file has a specific role.

```
python-redis-app/  
├─ app.py  
├─ requirements.txt  
├─ Dockerfile  
├─ docker-compose.yml  
├─ Jenkinsfile  
├─ sonar-project.properties  
├─ tests/  
└─ README.md
```

app.py

Flask application entry point and route definitions.

Dockerfile

Container image build instructions and layer configuration.

Jenkinsfile

Declarative pipeline definition — all 10 stages.

sonar-project.properties

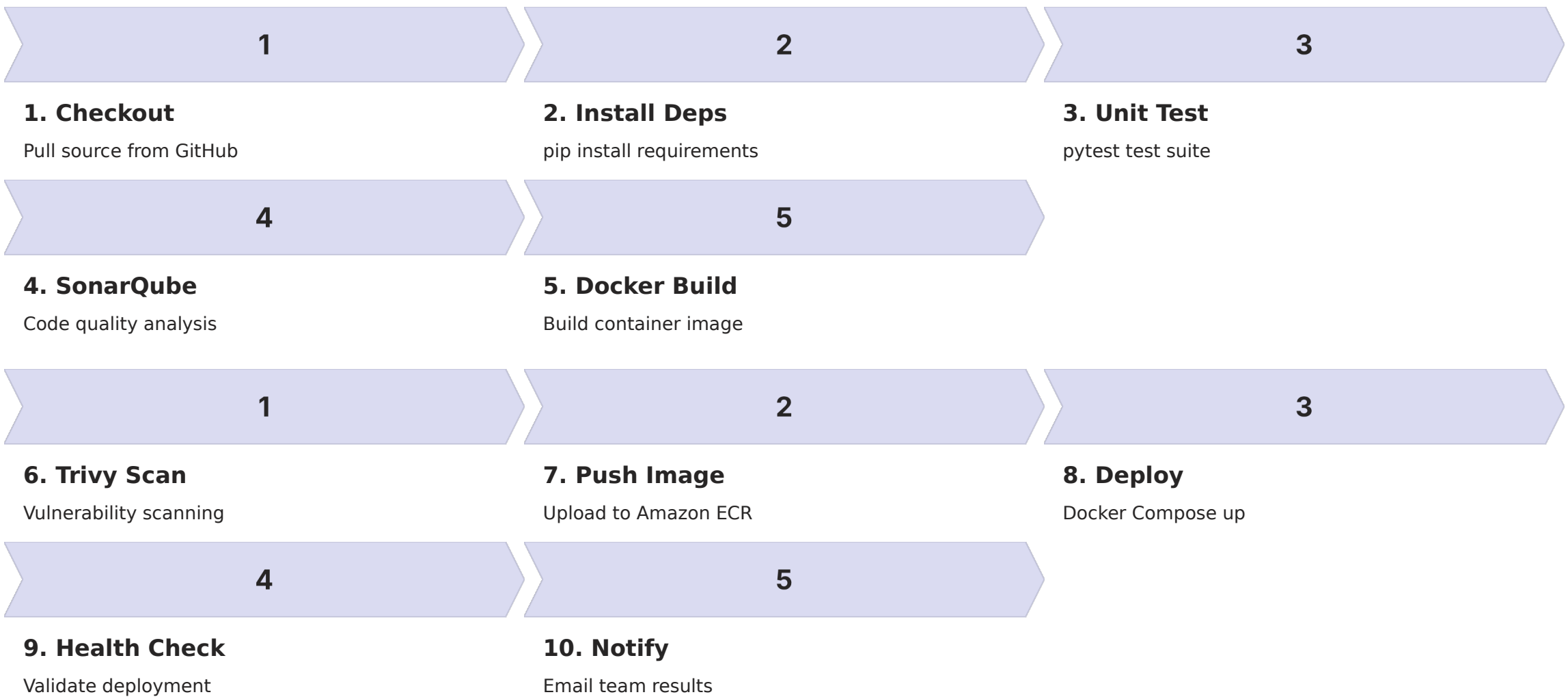
SonarQube project config, source paths, and quality gate settings.

tests/

pytest unit test suite for build validation.

Jenkins Pipeline Overview

The Jenkinsfile defines **10 automated stages** — from source checkout to team notification. Each stage is a quality gate.





Stage 1 - Source Code Checkout

GitHub Repository

Central source of truth for all application code and pipeline configuration.

Webhook Trigger

Every push to the target branch automatically triggers the Jenkins pipeline — zero manual intervention.

Branch Strategy

Feature branches merged to **main** trigger production pipeline. Protects release integrity.

Best Practices

Use **.gitignore** to exclude secrets. Never commit credentials. Tag releases for traceability.

Stage 2 - Python Build & Unit Testing

Shift-Left Testing means catching bugs as early as possible — before they reach staging or production.

01

pip install

Install all dependencies from **requirements.txt** into an isolated virtual environment.

02

pytest Execution

Run the full unit test suite. Jenkins captures test reports and marks build as passed or failed.

03

Build Validation

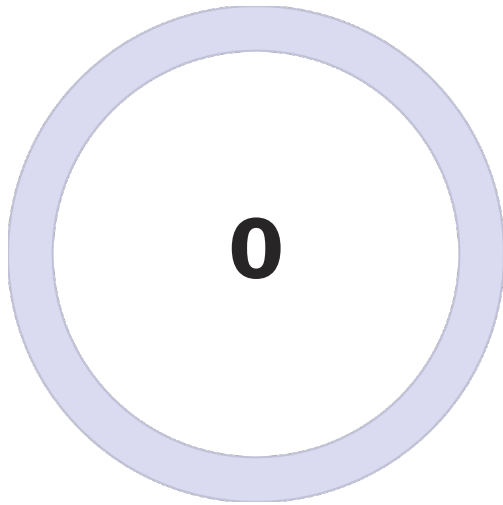
Pipeline halts immediately on test failure — broken code never advances downstream.

- ✔ Shift-Left Testing reduces the cost of fixing bugs by up to 10x compared to catching them in production.



Stage 3 - SonarQube Code Quality

SonarQube enforces a **Quality Gate** — the pipeline fails if code does not meet defined thresholds. No exceptions.

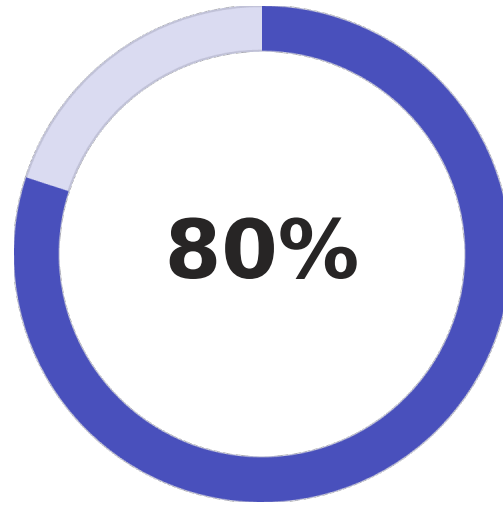


Bugs Allowed

Quality Gate requires zero critical bugs to pass.

Bugs & Vulnerabilities

Detects logic errors and security weaknesses in source code.

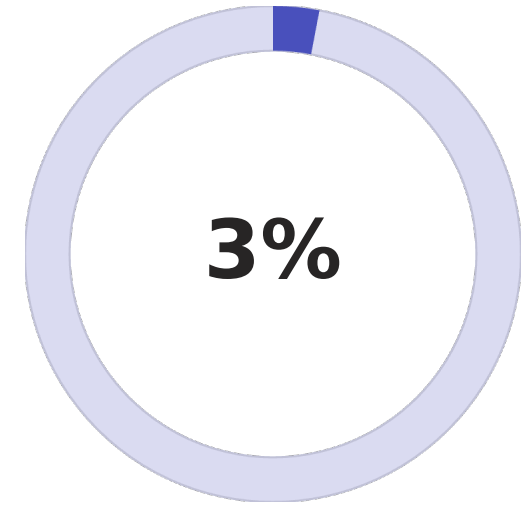


Coverage Target

Minimum test coverage required for gate approval.

Code Smells

Flags maintainability issues and technical debt before they compound.



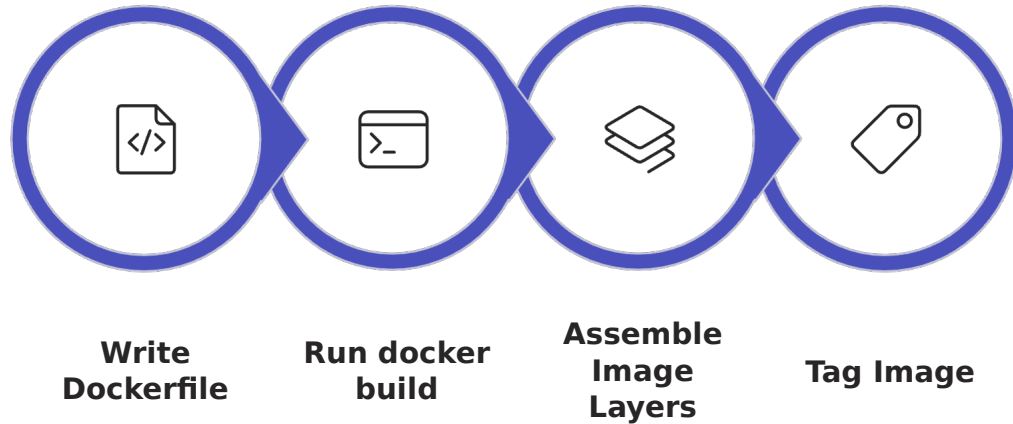
Max Duplication

Code duplication must stay below threshold.

Coverage & Duplication

Measures test coverage and identifies copy-paste code patterns.

Stage 4 - Docker Image Build



Build Best Practices

→ Use slim base images

Alpine or python:slim reduces attack surface and image size.

→ Layer caching

Copy requirements.txt before app code to maximize cache hits.

→ Semantic tagging

Tag images with Git commit SHA and build number for full traceability.

→ Non-root user

Run containers as a non-privileged user for security hardening.



Stage 5 - Trivy Security Scan

Trivy scans the built Docker image for known **CVEs (Common Vulnerabilities and Exposures)** before it ever reaches the registry.

Critical

Build **fails immediately**. Image is blocked from ECR push. Must be remediated before pipeline can proceed.

High

Build fails by default. Configurable threshold — teams can set acceptable risk tolerance per policy.

Medium / Low

Logged and reported. Pipeline continues. Tracked for scheduled remediation sprints.



Security must be automated — manual reviews are too slow and inconsistent for modern release cadences. Shift security left alongside testing.

Key Takeaways & Next Steps

This capstone delivers a **production-grade, fully automated CI/CD pipeline** — the same architecture used by enterprise DevOps teams worldwide



Set Up Your Repository

Clone the starter repo, review all files, and configure your GitHub webhook to trigger Jenkins.



Provision Infrastructure

Stand up Jenkins, SonarQube, and configure Amazon ECR credentials and Docker Compose environment.



Run Your First Pipeline

Push a commit, watch all 10 stages execute, review SonarQube dashboard and Trivy scan report.



Demonstrate & Document

Present your working pipeline, explain each architectural decision, and add it to your portfolio.

You are no longer a student — you are a DevOps engineer who has shipped a real enterprise pipeline.